



南開大學
Nankai University

计算机学院
数据安全实验报告

频率隐藏 OPE

姓名：王一诺
学号：2312331
专业：计算机科学与技术

2026 年 6 月 4 日

目录

1	实验要求	2
2	复现频率隐藏 OPE 方案	2
2.1	环境准备	2
2.2	编写 Server 端程序	3
2.3	编译生成动态链接库	12
2.4	导入 MySQL	13
2.5	编写 Client 端程序并测试	14
3	相同数值多次插入测试与编码树分裂、编码更新的观察	16
3.1	代码修改	16
3.2	运行测试	20
3.3	观察与分析	21
3.4	结论	24
4	结论与实践心得	24

1 实验要求

- 参照教材 6.3.3 FH-OPE 实现, 完成频率隐藏 OPE 方案的复现。
- 尝试在 client.py 中修改, 完成不断插入相同数值多次的测试, 观察编码树分裂和编码更新等情况。

2 复现频率隐藏 OPE 方案

2.1 环境准备

2.1.1 MySQL 的安装与配置

使用命令安装 MYSQL 后, 首先通过如下指令以 root 身份登陆数据库, 如图所示:

```
1 sudo mysql -u root -p
```

```
wyn@ubuntu:~/Desktop$ sudo mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 5.7.33-0ubuntu0.16.04.1 (Ubuntu)

Copyright (c) 2000, 2021, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

接下来可以通过 create user 指令创建用户, 结构如下:

```
1 CREATE USER 'username'@'host' IDENTIFIED BY 'password';
```

创建完成后, 可以通过 grant 命令授予用户不同的权限。例如创建一个用户名为 user, 不限制 host, 密码为 123456 的账户, 并将所有数据库上的所有权限授予用户 user:

```
mysql> CREATE USER 'user'@'%' IDENTIFIED BY '123456';
Query OK, 0 rows affected (0.00 sec)

mysql> grant all on *.* to 'user'@'%';
Query OK, 0 rows affected (0.00 sec)
```

然后使用指令创建数据库。

2.1.2 Python3 环境

Ubuntu16.04 之后已经默认包含 python3 环境, 无需而外安装。但是并没有 pip 工具, 所以需要
通过如下命令安装, 同时安装 client 需要用到的以及用于连接数据库的 pymysql:

```
1 [sudo] apt install python3-pip
2 pip3 install pycryptodome pymysql
3
```

2.2 编写 Server 端程序

根据 FH-OPE 描述，编写 Node.h 文件：

```

1  #pragma once
2  #include <vector>
3  #include <map>
4  #include <string>
5  using namespace std;
6
7  class Node
8  {
9      public:
10     int type;          // 用来区分InternalNode和LeafNode
11     int parent_index; // 记录当前结点是父结点的第几个孩子
12     Node *parent = NULL;
13     virtual void rebalance(){};
14     virtual long long insert(int pos, string cipher) { return 0; }; // 插入新的密文
15     virtual long long search(int pos) { return 0; }; // 查找pos对应的code
16 };
17
18 class InternalNode : public Node
19 {
20     public:
21     std::vector<int> child_num; // 子节点具有的加密值个数
22     std::vector<Node *> child; // 子节点指针
23
24     InternalNode();
25     void rebalance() override;
26     long long insert(int pos, string cipher) override;
27     long long search(int pos) override;
28     void insert_node(int index, Node *new_node); // 插入新的Node
29 };
30
31 class LeafNode : public Node
32 {
33     public:
34     std::vector<std::string> cipher; // 密文
35     std::vector<long long> encoding; // 编码
36     LeafNode *left_bro = NULL;      // 左兄弟节点
37     LeafNode *right_bro = NULL;     // 右兄弟节点
38     long long lower = -1;
39     long long upper = -1;
40
41     LeafNode();
42     long long Encode(int pos);
43     void rebalance() override;
44     long long insert(int pos, string cipher) override;
45     long long search(int pos) override;
46 };

```

```

47
48     const int M = 128;
49     extern Node *root;
50     extern long long start_update; // 更新区间的左端点
51     extern long long end_update;  // 更新区间的右端点
52     extern std::map<string, long long> update;
53
54     void root_initial();           // 初始化
55     long long get_update(string cipher); // 根据密文获取对应的更新后的code
56

```

在同文件夹下编写对应的 FH-OPE.cpp 文件:

```

1     #include "Node.h"
2     #include <array>
3     #include <math.h>
4     #include <assert.h>
5     #include <vector>
6     #include <map>
7     #include <fstream>
8
9     Node* root = nullptr;
10    long long start_update = -1;
11    long long end_update = -1;
12    std::map<string, long long> update;
13
14    InternalNode::InternalNode()
15    {
16        this->type = 2;
17        this->parent_index = -1;
18        this->parent = NULL;
19    }
20
21    void InternalNode::insert_node(int index, Node *new_node)
22    {
23        this->child.insert(this->child.begin() + index, new_node);
24        if (new_node->type == 1)
25        {
26            // 如果新结点是LeafNode
27            this->child_num.insert(this->child_num.begin() + index, ((LeafNode
28            *)new_node)->cipher.size());
29            ((LeafNode *)new_node)->parent = this;
30        }
31        else
32        {
33            // 如果是InternalNode
34            int res = 0;
35            for (size_t i = 0; i < ((InternalNode *)new_node)->child_num.size(); i++)
36            {

```

```

36         res += ((InternalNode *)new_node)->child_num.at(i);
37     }
38     this->child_num.insert(this->child_num.begin() + index, res);
39     ((InternalNode *)new_node)->parent = this;
40 }
41
42 // 插入的新node改变了原有node位置, 因此需要重新记录parent_index和child_num
43 for (int i = 0; i < this->child.size(); i++)
44 {
45     this->child.at(i)->parent_index = i;
46     if (this->child.at(i)->type == 1)
47     {
48         LeafNode *tmp = (LeafNode *)this->child.at(i);
49         this->child_num.at(i) = tmp->cipher.size();
50     }
51 }
52 if (this->child.size() >= M)
53 {
54     this->rebalance();
55 }
56 }
57
58 void InternalNode::rebalance()
59 {
60     InternalNode *new_node = new InternalNode();
61     // 将当前结点的后半部分数据存入new_node
62     int middle = floor(this->child.size() * 0.5);
63     while (middle > 0)
64     {
65         new_node->child.insert(new_node->child.begin(), this->child.at(this->child.size() - 1));
66         new_node->child_num.insert(new_node->child_num.begin(),
this->child_num.at(this->child_num.size() - 1));
67         this->child.pop_back();
68         this->child_num.pop_back();
69         middle--;
70     }
71     for (int i = 0; i < new_node->child.size(); i++)
72     {
73         new_node->child.at(i)->parent_index = i;
74         new_node->child.at(i)->parent = new_node;
75     }
76
77     if (!this->parent)
78     {
79         // 如果当前结点是root结点
80         InternalNode *new_root = new InternalNode();
81         new_root->insert_node(0, this);
82         new_root->insert_node(1, new_node);
83         root = new_root;

```

```

84     }
85     else
86     {
87         int res = 0;
88         for (size_t i = 0; i < this->child_num.size(); i++)
89         {
90             res += this->child_num.at(i);
91         }
92         ((InternalNode *)this->parent)->child_num.at(this->parent_index) = res;
93         ((InternalNode *)this->parent)->insert_node(this->parent_index + 1, new_node);
94     }
95 }
96
97 long long InternalNode::insert(int pos, string cipher)
98 {
99     for (int i = 0; i < this->child.size(); i++)
100     {
101         if (pos > this->child_num.at(i))
102         {
103             pos = pos - this->child_num.at(i);
104         }
105         else
106         {
107             this->child_num.at(i)++;
108             return this->child.at(i)->insert(pos, cipher);
109         }
110     }
111     // 如果没有符合的, 则放到最后一个
112     this->child_num.back() = this->child_num.back()++;
113     return this->child.back()->insert(pos, cipher);
114 }
115
116 long long InternalNode::search(int pos)
117 {
118     int i = 0;
119     for (; i < this->child.size(); i++)
120     {
121         if (pos < this->child_num.at(i))
122         {
123             return this->child.at(i)->search(pos);
124         }
125         else
126         {
127             pos = pos - this->child_num.at(i);
128         }
129     }
130     return 0;
131 }
132

```

```

133 LeafNode::LeafNode()
134 {
135     this->type = 1;
136     this->parent_index = -1;
137     this->parent = NULL;
138 }
139
140 void Recode(vector<LeafNode *> node_list)
141 {
142     long long left_bound = node_list.at(0)->lower;
143     long long right_bound = node_list.back()->upper;
144     int total_cipher_num = 0;
145
146     for (size_t i = 0; i < node_list.size(); i++)
147     {
148         total_cipher_num += node_list.at(i)->cipher.size();
149     }
150
151     if ((right_bound - left_bound) > total_cipher_num)
152     {
153         // 如果当前的更新区间，足以包含待放的pos
154         start_update = left_bound;
155         end_update = right_bound;
156         // 计算间隔量，使code均匀分布
157         long long frag = floor((right_bound - left_bound) / total_cipher_num);
158         assert(frag >= 1);
159         long long cd = left_bound;
160         for (size_t i = 0; i < node_list.size(); i++)
161         {
162             node_list.at(i)->lower = cd;
163             for (int j = 0; j < node_list.at(i)->encoding.size(); j++)
164             {
165                 node_list.at(i)->encoding.at(j) = cd;
166                 update.insert(make_pair(node_list.at(i)->cipher.at(j), cd));
167                 cd = cd + frag;
168             }
169             node_list.at(i)->upper = cd;
170         }
171         node_list.back()->upper = right_bound;
172     }
173     else
174     {
175         // 若不足以包含，则继续向左兄弟结点和右兄弟结点扩展更新区间
176         if (node_list.at(0)->left_bro)
177         {
178             // 如果左兄弟存在，则加入更新列表
179             node_list.insert(node_list.begin(), node_list.at(0)->left_bro);
180         }
181     }

```



```

182     if (node_list.back()->right_bro)
183     {
184         // 如果右兄弟存在，则加入更新列表
185         node_list.push_back(node_list.back()->right_bro);
186     }
187     else
188     {
189         // 扩展最后一个结点的大小
190         node_list.back()->upper = node_list.back()->upper * 2;
191         if (node_list.back()->upper >= pow(2, 60))
192             node_list.back()->upper = pow(2, 60);
193     }
194     Recode(node_list);
195 }
196 }
197
198 long long LeafNode::Encode(int pos)
199 {
200     long long left = this->lower;
201     long long right = this->upper;
202
203     if (pos > 0)
204     {
205         left = this->encoding.at(pos - 1);
206     }
207     if (pos < this->encoding.size() - 1)
208     {
209         right = this->encoding.at(pos + 1);
210     }
211     if (floor(right - left) < 2)
212     {
213         // 如果该区间已经没有位置，则需要recode
214         std::vector<LeafNode *> node_list;
215         node_list.push_back(this);
216         Recode(node_list);
217         // 返回特殊值0，标志已经调整，需要更新数据库
218         return 0;
219     }
220     else
221     {
222         // 否则直接以left和right的平均值向上取整作为新的code
223         unsigned long long re = right;
224         long long frag = (right - left) / 2;
225         re = re - frag;
226         this->encoding.at(pos) = re;
227         return this->encoding.at(pos);
228     }
229 }
230

```

```

231 void LeafNode::rebalance()
232 {
233     LeafNode *new_node = new LeafNode();
234     // 将本结点后半部分数据放到new_node
235     int middle = floor(this->cipher.size() * 0.5);
236     while (middle > 0)
237     {
238         new_node->cipher.insert(new_node->cipher.begin(), this->cipher.back());
239         new_node->encoding.insert(new_node->encoding.begin(), this->encoding.back());
240         this->encoding.pop_back();
241         this->cipher.pop_back();
242         middle--;
243     }
244     // 将新结点连接在this结点右边
245     new_node->lower = new_node->encoding.at(0);
246     new_node->upper = this->upper;
247     this->upper = new_node->encoding.at(0);
248     if (this->right_bro)
249     {
250         this->right_bro->left_bro = new_node;
251     }
252     new_node->right_bro = this->right_bro;
253     this->right_bro = new_node;
254     new_node->left_bro = this;
255
256     if (!this->parent)
257     {
258         // 如果this是root结点, 则创建一个新的root
259         InternalNode *new_root = new InternalNode();
260         new_root->insert_node(0, this);
261         new_root->insert_node(1, new_node);
262         root = new_root;
263     }
264     else
265     {
266         ((InternalNode *)this->parent)->child_num.at(this->parent_index) = this->cipher.size();
267         ((InternalNode *)this->parent)->insert_node(this->parent_index + 1, new_node);
268     }
269 }
270
271 long long LeafNode::insert(int pos, string cipher)
272 {
273     // 将密文插入到对应的pos
274     this->cipher.insert(this->cipher.begin() + pos, cipher);
275     this->encoding.insert(this->encoding.begin() + pos, -1);
276     long long cd = this->Encode(pos);
277     if (this->cipher.size() >= M)
278     {
279         this->rebalance();

```

```

280     }
281     return cd;
282 }
283
284 long long LeafNode::search(int pos)
285 {
286     return this->encoding.at(pos);
287 }
288
289 void root_initial()
290 {
291     root = new LeafNode();
292     ((LeafNode *)root)->lower = 0;
293     ((LeafNode *)root)->upper = pow(2, 62);
294     update.clear();
295     start_update = -1;
296     end_update = -1;
297 };
298
299 long long get_update(string cipher)
300 {
301     if (update.count(cipher) > 0)
302     {
303         return update[cipher];
304     }
305     return 0;
306 }
307

```

为能够在 MySQL 中使用，在 UDF.cpp 文件中编写 udf 函数：

```

1  #include "Node.h"
2  #include "mysql/mysql.h"
3  #include <string.h>
4
5  extern "C"
6  {
7      // 插入
8      bool FHInsert_init(UDF_INIT *initid, UDF_ARGS *args, char *message);
9      long long FHInsert(UDF_INIT *initid, UDF_ARGS *args, char *is_null, char *error);
10     // 搜索
11     bool FHSearch_init(UDF_INIT *const initid, UDF_ARGS *const args, char *const message);
12     long long FHSearch(UDF_INIT *const initid, UDF_ARGS *const args,
13         char *const result, unsigned long *const length, char *const is_null, char *const error);
14
15     // 更新
16     bool FHUpdate_init(UDF_INIT *initid, UDF_ARGS *args, char *message);
17     long long FHUpdate(UDF_INIT *initid, UDF_ARGS *args, char *is_null, char *error);
18     // 更新范围

```

```

18     bool FHStart_init(UDF_INIT *initid, UDF_ARGS *args, char *message);
19     long long FHStart(UDF_INIT *initid, UDF_ARGS *args, char *is_null, char *error);
20     bool FHEnd_init(UDF_INIT *initid, UDF_ARGS *args, char *message);
21     long long FHEnd(UDF_INIT *initid, UDF_ARGS *args, char *is_null, char *error);
22 }
23
24 static char * getba(UDF_ARGS *const args, int i, double &len)
25 {
26     len = args->lengths[i];
27     return args->args[i];
28 }
29
30 /*插入*/
31 long long FHInsert(UDF_INIT *initid, UDF_ARGS *args, char *is_null, char *error)
32 {
33     int pos = *(int *) (args->args[0]);
34     double keyLen;
35     char *const keyBytes = getba(args, 1, keyLen);
36     const std::string cipher = std::string(keyBytes, keyLen);
37     long long start_update = -1;
38     long long end_update = -1;
39     update.clear();
40     long long re = root->insert(pos, cipher);
41     return re;
42 }
43
44 bool FHInsert_init(UDF_INIT *initid, UDF_ARGS *args, char *message)
45 {
46     start_update = -1;
47     end_update = -1;
48     update.clear();
49     if (root == nullptr)
50     {
51         root_initial();
52     }
53     return 0;
54 }
55
56 /*搜索*/
57 long long FHSearch(UDF_INIT *const initid, UDF_ARGS *const args, char *const result, unsigned
long *const length, char *const is_null, char *const error)
58 {
59     int pos = *(int *) (args->args[0]);
60     if (pos < 0)
61         return 0;
62     return root->search(pos);
63 }
64 bool FHSearch_init(UDF_INIT *const initid, UDF_ARGS *const args, char *const message)
65 {

```

```
66     return 0;
67 }
68
69 long long FHUpdate(UDF_INIT *initid, UDF_ARGS *args, char *is_null, char *error)
70 {
71     double keyLen;
72     char *const keyBytes = getba(args, 0, keyLen);
73     const std::string cipher = std::string(keyBytes, keyLen);
74     long long update_code = get_update(cipher);
75     return update_code;
76 }
77
78 bool FHUpdate_init(UDF_INIT *initid, UDF_ARGS *args, char *message)
79 {
80     return 0;
81 }
82
83 long long FHStart(UDF_INIT *initid, UDF_ARGS *args, char *is_null, char *error)
84 {
85     return start_update;
86 }
87
88 bool FHStart_init(UDF_INIT *initid, UDF_ARGS *args, char *message)
89 {
90     return 0;
91 }
92
93 long long FHEnd(UDF_INIT *initid, UDF_ARGS *args, char *is_null, char *error)
94 {
95     return end_update;
96 }
97
98 bool FHEnd_init(UDF_INIT *initid, UDF_ARGS *args, char *message)
99 {
100     return 0;
101 }
102
```

2.3 编译生成动态链接库

使用如下命令编译生成动态链接库 libzidx.so:

```
1 g++ -std=c++11 -shared -fPIC UDF.cpp FH-OPE.cpp -I/usr/include/mysql -lcrypto -o libfhope.so
2
```

然后将其拷贝到 MySQL 的文件夹:

```
1 sudo cp libfhope.so /usr/lib/mysql/plugin/
2
```

2.4 导入 MySQL

编写 sql 文件:

```
1  -- 如果原来有同名表, 则删除
2  drop table if exists example;
3  -- 创建名为example的表
4  create table example
5  (
6      encoding    bigint,
7      ciphertext  varchar(512)
8  );
9
10 -- 删除已有的函数
11 drop function if exists FHInsert;
12 drop function if exists FHSearch;
13 drop function if exists FHUpdate;
14 drop function if exists FHStart;
15 drop function if exists FHEnd;
16
17 -- 创建函数
18 create function FHInsert RETURNS INTEGER SONAME 'libfhope.so';
19 create function FHSearch RETURNS INTEGER SONAME 'libfhope.so';
20 create function FHUpdate RETURNS INTEGER SONAME 'libfhope.so';
21 create function FHStart RETURNS INTEGER SONAME 'libfhope.so';
22 create function FHEnd RETURNS INTEGER SONAME 'libfhope.so';
23
24 -- 创建插入数据的存储过程
25 drop procedure if exists pro_insert;
26 delimiter $$
27 create procedure pro_insert(IN pos int, IN ct varchar(512))
28 BEGIN
29     DECLARE i BIGINT default 0;
30     SET i = FHInsert(pos, ct);
31     insert into example values (i, ct);
32     if i = 0 then
33         -- 树结构中更新了编码, 同步更新数据库中的信息
34         update example
35         set encoding = FHUpdate(ciphertext)
36         where (encoding >= FHStart() and encoding < FHEnd())
37             or (encoding = 0);
38     end if;
39 END $$
40 delimiter ;
41
```

接下来, 登陆并切换数据库:

```
mysql -u<用户名> -p<密码>
use <数据库名>
```

```
wyn@ubuntu:~/Desktop/FH-OPE$ mysql -uuser -p123456
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 6
Server version: 5.7.33-0ubuntu0.16.04.1 (Ubuntu)

Copyright (c) 2000, 2021, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> use test_db
Database changed
mysql> █
```

然后导入 sql 文件：

```
source <sql文件绝对地址>
```

```
mysql> source /home/wyn/Desktop/FH-OPE/load.sql
Query OK, 0 rows affected, 1 warning (0.00 sec)

Query OK, 0 rows affected (0.02 sec)

Query OK, 0 rows affected, 1 warning (0.00 sec)

Query OK, 0 rows affected, 1 warning (0.01 sec)

Query OK, 0 rows affected, 1 warning (0.00 sec)

Query OK, 0 rows affected, 1 warning (0.00 sec)

Query OK, 0 rows affected, 1 warning (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

Query OK, 0 rows affected (0.00 sec)

Query OK, 0 rows affected, 1 warning (0.00 sec)

Query OK, 0 rows affected (0.00 sec)
```

2.5 编写 Client 端程序并测试

编写 Client 部分代码，存入 client.py：

```
import pymysql
import random
from Crypto.Cipher import AES
from Crypto.Random import get_random_bytes
from Crypto.Util.Padding import pad, unpad
import base64
```

```

8 local_table = {}
9 key = get_random_bytes(16)
10 base_iv = get_random_bytes(16)
11
12 def AES_ENC(plaintext, iv):
13     # AES加密
14     aes = AES.new(key, AES.MODE_CBC, iv=iv)
15     padded_data = pad(plaintext, AES.block_size, style='pkcs7')
16     ciphertext = aes.encrypt(padded_data)
17     return ciphertext
18
19 def AES_DEC(ciphertext, iv):
20     # AES解密
21     aes = AES.new(key, AES.MODE_CBC, iv=iv)
22     padded_data = aes.decrypt(ciphertext)
23     plaintext = unpad(padded_data, AES.block_size, style='pkcs7')
24     return plaintext
25
26 def Random_Encrypt(plaintext):
27     # 随机生成iv来保证加密结果的随机性
28     iv = get_random_bytes(16)
29     ciphertext = AES_ENC(iv + AES_ENC(plaintext.encode('utf-8'), iv), base_iv)
30     ciphertext = base64.b64encode(ciphertext)
31     return ciphertext.decode('utf-8')
32
33 def Random_Decrypt(ciphertext):
34     plaintext = AES_DEC(base64.b64decode(ciphertext.encode('utf-8')), base_iv)
35     plaintext = AES_DEC(plaintext[16:], plaintext[:16])
36     return plaintext.decode('utf-8')
37
38 def CalPos(plaintext):
39     # 插入plaintext, 返回对应的Pos
40     presum = sum([v for k, v in local_table.items() if k < plaintext])
41     if plaintext in local_table:
42         local_table[plaintext] += 1
43     return random.randint(presum, presum + local_table[plaintext] - 1)
44     else:
45         local_table[plaintext] = 1
46     return presum
47
48 def GetLeftPos(plaintext):
49     return sum([v for k, v in local_table.items() if k < plaintext])
50
51 def GetRightPos(plaintext):
52     return sum([v for k, v in local_table.items() if k <= plaintext])
53
54 def Insert(plaintext):
55     ciphertext = Random_Encrypt(plaintext)
56     # 连接数据库

```



```

57     conn = pymysql.connect(host='localhost', user='user', passwd='123456', database='test_db')
58     cur = conn.cursor()
59     cur.execute("call pro_insert(%s, %s)", (CalPos(plaintext), ciphertext))
60     conn.commit()
61     conn.close()
62
63 def Search(left, right):
64     # 搜索[left,right]中的信息
65     left_pos = GetLeftPos(left)
66     right_pos = GetRightPos(right)
67     # 连接数据库
68     conn = pymysql.connect(host='localhost', user='user', passwd='123456', database='test_db')
69     cur = conn.cursor()
70     cur.execute(
71         "select ciphertext from example where encoding >= FHSearch(%s) and encoding < FHSearch(%s)",
72         (left_pos, right_pos)
73     )
74     rest = cur.fetchall()
75     for x in rest:
76         print("ciphertext: {} plaintext: {}".format(x[0], Random_Decrypt(x[0])))
77
78 if __name__ == '__main__':
79     # 插入明文，同时设置了一部分重复的内容
80     for ciphertext in ['apple', 'pear', 'banana', 'orange', 'cherry', 'apple', 'cherry',
81 'orange']:
82         Insert(ciphertext)
83
84     # 假设我们搜索b和p之间的数据
85     Search('b', 'p')
86

```

然后通过 python3 指令可直接执行 py 文件，如图所示，实验打印了密文及解密后的明文：

```

wyn@ubuntu:~/Desktop/FH-OPE$ python3 client.py
ciphertext: 1CcIfhkj5iJtPm56xp7+lIG2TzjaPi887wG/EygplYnnJSv6VwD/tSojFoRh92kv plaintext: banana
ciphertext: hkSVMQGGwLzzcEhxf6XyHJ1GEF6drTfMvc902McMBuTjK/qS87+d53mVyFlqtA plaintext: orange
ciphertext: 1uS8J3Casl71faUhyt+4xH/XDfpDKa32879G0tWXqgSChOoHGG5vnQpJUwtP+IDZ plaintext: cherry
ciphertext: ahlhgrifspXHA/xiQbF/hCw/aItehEivkocxfMu0VKOn7BoayrrRFnIXJa4x6MF plaintext: cherry
ciphertext: vVZ6suXxgcpBw3eXYN4117DrLxQXDzi/YDO/dXbSRmAsQNPOA8YYIWRK7oIzoCb7 plaintext: orange
wyn@ubuntu:~/Desktop/FH-OPE$

```

3 相同数值多次插入测试与编码树分裂、编码更新的观察

3.1 代码修改

基础复现部分的 client.py 只插入了少量不同明文，能够验证 FH-OPE 插入、加密和范围查询流程，但不足以稳定观察编码树分裂和编码更新。原因在于本实验实现中叶子结点容量阈值为 $M=128$ ，并且初始编码区间很大，若继续采用随机位置插入，140 次左右的插入虽然可能触发叶子结点分裂，但不

一定触发 Recode 和 FHUpdate。因此在第二部分实验中，我将 client.py 改为专门用于压力测试的版本，主要目标如下：

- 连续插入相同明文 apple，观察客户端频数表如何记录重复值；
- 通过固定插入策略不断压缩局部编码间隔，使 FHInsert 更容易返回 0，从而触发编码重新分配；
- 展开原存储过程 pro_insert 的核心逻辑，在客户端显式打印 FHInsert、FHStart、FHEnd 和 FHUpdate 的执行情况；
- 在第 128 次插入附近观察叶子结点容量达到阈值后的树结构调整现象；
- 修正相同明文查询时右边界等于总记录数导致的查询边界问题，保证最终能够查回全部重复明文。

首先修改 CalPos 函数。原始版本在同一明文对应的重复区间中随机选择插入位置，这能够体现频率隐藏思想，但不利于在较短实验中稳定触发编码更新。因此新增 strategy 参数，其中 left_edge 策略总是把新密文插入到当前相同明文区间的最左端，不断压缩左端局部编码空间。

```

1 def CalPos(plaintext, strategy='random'):
2     # 插入plaintext, 返回对应的Pos
3     presum = sum([v for k, v in local_table.items() if k < plaintext])
4     old_count = local_table.get(plaintext, 0)
5     new_count = old_count + 1
6     local_table[plaintext] = new_count
7
8     if old_count == 0:
9         return presum
10
11    if strategy == 'left_edge':
12        # 压力测试模式：总是插入到相同明文区间左端，快速耗尽局部编码间隔。
13        return presum
14    if strategy == 'right_edge':
15        return presum + new_count - 1
16    if strategy == 'middle':
17        return presum + new_count // 2
18
19    return random.randint(presum, presum + new_count - 1)
20

```

然后新增数据库快照和表内容打印函数，用于比较一次插入前后已有记录的 encoding 是否变化。这里以密文为键记录插入前的编码值，再与插入后的编码值进行比较。如果旧密文对应的编码发生变化，说明本轮执行了编码更新。

```

1 def Snapshot(cur):
2     cur.execute("select encoding, ciphertext from example order by encoding, ciphertext")
3     rows = cur.fetchall()
4     result = {}
5     for encoding, ciphertext in rows:
6         result[ciphertext] = encoding
7     return result
8

```

```

9 def PrintTable(cur, title, max_rows=20):
10     cur.execute("select encoding, ciphertext from example order by encoding, ciphertext")
11     rows = cur.fetchall()
12     print("")
13     print("[{}] total rows = {}".format(title, len(rows)))
14     for index, row in enumerate(rows[:max_rows], start=1):
15         encoding = row[0]
16         ciphertext = row[1]
17         try:
18             plaintext = Random_Decrypt(ciphertext)
19         except Exception:
20             plaintext = "<cannot decrypt: old key or dirty table>"
21         print("{:03d}: encoding={}, plaintext={}, ciphertext={}...".format(
22             index, encoding, plaintext, ciphertext[:24]
23         ))
24     if len(rows) > max_rows:
25         print("... only first {} rows are shown".format(max_rows))
26

```

核心修改是将原来在 MySQL 存储过程 `pro_insert` 中封装的逻辑展开到客户端中执行。这样可以直接观察 `FHInsert` 的返回值：若返回非零编码，说明本次插入可以直接在相邻编码之间生成新编码；若返回 0，说明局部编码空间不足，服务端需要执行 `Recode`，随后数据库中受影响区间的旧记录也要通过 `FHUpdate(ciphertext)` 更新编码。

```

1 def InsertWithTrace(plaintext, step=None, print_table=False, strategy='random'):
2     ciphertext = Random_Encrypt(plaintext)
3     pos = CalPos(plaintext, strategy=strategy)
4     conn = OpenConn()
5     cur = conn.cursor()
6
7     before = Snapshot(cur)
8     if step is None:
9         label = "insert"
10    else:
11        label = "step {}".format(step)
12    print("")
13    print("==== {}: plaintext={}, pos={}, local_table={} =====".format(
14        label, plaintext, pos, local_table
15    ))
16    print("insert strategy: {}".format(strategy))
17
18    # 展开 pro_insert 的核心逻辑，便于观察 FHInsert 和 FHUpdate。
19    cur.execute("select FHInsert(%s, %s)", (pos, ciphertext))
20    inserted_encoding = cur.fetchone()[0]
21    print("FHInsert returned: {}".format(inserted_encoding))
22
23    cur.execute("insert into example values (%s, %s)", (inserted_encoding, ciphertext))
24
25    if inserted_encoding == 0:

```

```

26     # 返回 0 表示触发 Recode, 需要更新数据库中受影响区间的 encoding。
27     cur.execute("select FHStart(), FHEnd()")
28     start_update, end_update = cur.fetchone()
29     print("[RECODE] update interval: [{}, {}]".format(start_update, end_update))
30
31     cur.execute(
32         "select count(*) from example "
33         "where (encoding >= FHStart() and encoding < FHEnd()) or encoding = 0"
34     )
35     affected = cur.fetchone()[0]
36     print("[RECODE] rows to refresh with FHUpdate: {}".format(affected))
37
38     cur.execute(
39         "update example "
40         "set encoding = FHUpdate(ciphertext) "
41         "where (encoding >= FHStart() and encoding < FHEnd()) or encoding = 0"
42     )
43     print("[RECODE] FHUpdate executed")
44 else:
45     print("[NORMAL] no FHUpdate is needed for this insert")
46
47     conn.commit()
48

```

在提交后, 继续比较插入前后的快照, 并输出已有记录的编码变化。该部分输出是判断 FHUpdate 是否真正改变数据库中旧记录编码的直接证据。

```

1     after = Snapshot(cur)
2     changed = []
3     for old_ciphertext, old_encoding in before.items():
4         new_encoding = after.get(old_ciphertext)
5         if new_encoding is not None and new_encoding != old_encoding:
6             changed.append((old_encoding, new_encoding, old_ciphertext))
7
8     if changed:
9         print("[ENCODING UPDATED] existing rows changed: {}".format(len(changed)))
10        for old_encoding, new_encoding, old_ciphertext in changed[:10]:
11            print(" {} -> {}, ciphertext={}\...".format(
12                old_encoding, new_encoding, old_ciphertext[:24]
13            ))
14    else:
15        print("[ENCODING UPDATED] no existing row changed")
16

```

最后修改测试入口, 连续插入 140 次相同明文 apple。选择 140 次是因为其超过了叶子结点阈值 M=128, 既可以观察编码更新, 也可以在第 128 次附近观察编码树分裂相关现象。

```

1 def RepeatedInsertTest(value='apple', times=140, strategy='left_edge'):
2     random.seed(2026)

```

```
3 print("==== Repeated insert test ====")
4 print("value={}, times={}, strategy={}".format(value, times, strategy))
5 print("strategy left_edge is used to force recode/update in a short run")
6 for i in range(1, times + 1):
7     # M = 128, 接近第 128 次时最容易观察到叶子结点分裂相关现象。
8     verbose = i <= 5 or i in [60, 61, 62, 63, 64, 126, 127, 128, 129, times]
9     if i == 128:
10         print("")
11         print("[SPLIT EXPECTED] step 128 reaches M=128; LeafNode::rebalance should be
12         triggered in the UDF")
13         InsertWithTrace(value, step=i, print_table=verbose, strategy=strategy)
14
15 print("")
16 print("==== Search result for the repeated plaintext ====")
17 Search(value, value)
18
19 if __name__ == '__main__':
20     RepeatedInsertTest('apple', 140, strategy='left_edge')
```

3.2 运行测试

运行测试前，为避免 MySQL UDF 中全局编码树状态残留，应先清空 `example` 表。如果此前已经多次运行实验，最好重启 MySQL 服务并重新导入 SQL 文件，使服务端内存中的编码树从空树开始。

```
1 use test_db;
2 delete from example;
3 -- 如多次实验后现象异常，可重启 MySQL 后重新导入 load.sql
4 sudo service mysql restart
5
```

随后在 FH-OPE 目录中执行 `client.py`。由于终端输出较长，这里只展示一处截图：

```
[table after step 140] total rows = 140
001: encoding=0, plaintext=apple, ciphertext=Naoc0z2Noxb1gxXsotjGMMLs...
002: encoding=30340039594917024, plaintext=apple, ciphertext=awk1SoWL/n3xbObSNFzad3wq...
003: encoding=60680079189834048, plaintext=apple, ciphertext=MfPPQqWJCqQ87390x4wDkRG...
004: encoding=91020118784751072, plaintext=apple, ciphertext=YBxdp//9IHdDuv0CiIERf9ak...
005: encoding=121360158379668096, plaintext=apple, ciphertext=o7w0BeVmlk4ZjSAQxE09G0T...
006: encoding=151700197974585120, plaintext=apple, ciphertext=M/ycGmJXHpH6IXfjm+v0Ihds...
007: encoding=182040237569502144, plaintext=apple, ciphertext=9dJbWdQMCAY8k+LfhgTiMCDc...
008: encoding=212380277164419168, plaintext=apple, ciphertext=z+8QgFPQc3cAE2aHUs6TYmu/...
009: encoding=242720316759336192, plaintext=apple, ciphertext=8d0lkq/sHs/EQ4BRHtONWS/U...
010: encoding=273060356354253216, plaintext=apple, ciphertext=1TVGErGsah7Sr5oEbPF+f5Gm...
011: encoding=303400395949170240, plaintext=apple, ciphertext=QtSCJpLpGoCXC2mxie3Y1k36...
012: encoding=333740435544087264, plaintext=apple, ciphertext=fuPRUsEgav6m9fxyNEX4F9if...
013: encoding=364080475139004288, plaintext=apple, ciphertext=vTdyKn9A4sSMhFvG83V3AxoL...
014: encoding=394420514733921312, plaintext=apple, ciphertext=e5HSe5AiPIT0nKnJ+MUPhLs/...
015: encoding=424760554328838336, plaintext=apple, ciphertext=F/K4zU75CwmNIyAiMOWsuFGT...
016: encoding=45510059323755360, plaintext=apple, ciphertext=snDPb5D5HfTYTWuA7w9g58W1...
017: encoding=485440633518672384, plaintext=apple, ciphertext=hy8M7Oso+bmWzZ1wL5a77apY...
018: encoding=515780673113589408, plaintext=apple, ciphertext=2gqTmWkH0LseeC2SPNGk6rJM...
019: encoding=546120712708506432, plaintext=apple, ciphertext=fM0yJRB5+ikKUJzDkrmYNkcI...
020: encoding=576460752303423456, plaintext=apple, ciphertext=wEm0u0eVHX55fSPIIE8wPG2...
... only first 20 rows are shown
[gap stats after step 140] min encoding gap = 30340039594917024, between row 1 and row 2

===== Search result for the repeated plaintext =====
Search(apple, apple) returned 140 rows
ciphertext: Naoc0z2Noxb1gxXsotjGMMLsuhi/vhpqhyMTe/Z/pYYimnbvgMhP7qHmilSTpeFd plaintext: apple
ciphertext: awk1SoWL/n3xbObSNFzad3wqnpNwBT0YgwS1SsEWQceJ8fXbMBsC0HNSUUhRkw+g plaintext: apple
ciphertext: MfPPQqWJCqQ87390x4wDkRGTAxy+VLm1Bs5ii+VzdkSEQQ9nLDZJtYz45MJxQV2 plaintext: apple
ciphertext: YBxdp//9IHdDuv0CiIERf9ak9xkJJz/1GoKEqVy457cExrAygBY6yRue+Z2jjhrq plaintext: apple
ciphertext: o7w0BeVmlk4ZjSAQxE09G0TvwQk28WEdPs8Jkpze3e/y7B2I4wKgeD9KH7W0j plaintext: apple
ciphertext: M/ycGmJXHpH6IXfjm+v0IhdsC4QA3IqENgUf76MhODwrZRe3yg3MbrFzJ330F5gj plaintext: apple
ciphertext: 9dJbWdQMCAY8k+LfhgTiMCDcfvGcGVf3gG0kFvXJ4CGT+8qfq9svDMGLuX6pmREI plaintext: apple
ciphertext: z+8QgFPQc3cAE2aHUs6TYmu//foS0gqKcV+UEb3PbzFBgx0D7m6b6M07bT13Erv2 plaintext: apple
ciphertext: 8d0lkq/sHs/EQ4BRHtONWS/UuLN0h292DeyXjtkCaBEKJ1rJ2XJ+Rhx+7Ftr01xz plaintext: apple
ciphertext: 1TVGErGsah7Sr5oEbPF+f5Gmfrn+Hmk9njFYp3AQkTyGGvS2M23S0k6ulb0EE21Z plaintext: apple
ciphertext: QtSCJpLpGoCXC2mxie3Y1k361YvXEJAGNaJZ7HwqDvjCXjzQmJiUkFjSX7mPyTiy plaintext: apple
ciphertext: fuPRUsEgav6m9fxyNEX4F9ifnQ4ZFSdp05Ua268n/05A0wcb6tBrJepek8XY7221 plaintext: apple
ciphertext: vTdyKn9A4sSMhFvG83V3AxoL+8eI+DXECnOxEGhJweGBuVzILp08pbc/7HeNDwC plaintext: apple
ciphertext: e5HSe5AiPIT0nKnJ+MUPhLs/oDFPZbc/Y74YFXu7LCn7g7pThyD3N7pSLzWdxk plaintext: apple
ciphertext: F/K4zU75CwmNIyAiMOWsuFGTdfp2YQ6+SblCp1WM+aRhtVZY6B+eRgRgMMEy8ls9 plaintext: apple
ciphertext: snDPb5D5HfTYTWuA7w9g58W1qVNjvSE+hYXRgf6AfcP08E6CnuK5Is8BsArwyOuF plaintext: apple
ciphertext: hy8M7Oso+bmWzZ1wL5a77apYBXPxdQwBitEP5kGqL8RY6kx3ln31UBZszt7ecyft plaintext: apple
ciphertext: 2gqTmWkH0LseeC2SPNGk6rJMEJVsQ/xp3TMJjyhaEv8CGmBmyOg4wboBVFpFMvAY plaintext: apple
```

本次运行中，统计得到的关键信息如下：

```
1 ===== step: 41
2 FHInsert returned: 0: 41
3 [RECODE] FHUpdate executed: 42
4 existing rows changed: 42
5 [SPLIT EXPECTED]: 1
6 Search(apple, apple) returned: 1
7 plaintext=apple: 141
8 ciphertext:: 140
9
```

其中，FHInsert 返回 0 出现 41 次，说明实验中多次触发了编码空间不足后的重新编码；FHUpdate 执行成功出现 42 次，说明数据库端已有记录的编码被多次刷新；最终输出了 140 条密文及其解密后的明文。

3.3 观察与分析

首先观察编码更新。在第 100 次插入时，终端输出如下：

```
1 ===== step 100: plaintext=apple, pos=0, local_table={'apple': 100} =====
2 insert strategy: left_edge
3 FHInsert returned: 0
```



```

4 [RECODE] update interval: [0, 4611686018427387904)
5 [RECODE] rows to refresh with FHUpdate: 100
6 [RECODE] FHUpdate executed
7 [ENCODING UPDATED] existing rows changed: 99
8   2375717039795926896 -> 2398076729582241760, ciphertext=gRZa4miCOH5CLBqNWQmscfK...
9   276701161105643280 -> 319621803257343720, ciphertext=hkU3a70gMaXw7MXmhOPTOUOM...
10  3260788093837546720 -> 3274297073083445480, ciphertext=J9P1dIqBG1kI8GRm/mEgPBdS...
11

```

这段输出说明，当第 100 次继续向相同明文区间左端插入时，局部编码间隔已经不足，FHInsert 无法直接返回一个新的非零编码，而是返回 0 作为触发重新编码的标志。随后程序调用 FHStart() 和 FHEnd() 得到更新区间，并对该区间内的 100 条记录执行 FHUpdate(ciphertext)。输出中的 old -> new 显示了旧记录编码的实际变化，这证明编码更新不是形式上的调用，而是确实改变了数据库中已有密文对应的顺序编码。

随后观察多次更新的连续性。第 101 次到第 128 次附近仍持续出现 FHInsert returned: 0 和 FHUpdate executed，说明在 left_edge 压力测试下，新的插入不断压缩局部区间，服务端需要反复进行重新编码。第 126 次到第 128 次的输出摘录如下：

```

1 ===== step 126: plaintext=apple, pos=0, local_table={'apple': 126} =====
2 insert strategy: left_edge
3 FHInsert returned: 0
4 [RECODE] update interval: [0, 4611686018427387904)
5 [RECODE] rows to refresh with FHUpdate: 126
6 [RECODE] FHUpdate executed
7 [ENCODING UPDATED] existing rows changed: 125
8
9 ===== step 127: plaintext=apple, pos=0, local_table={'apple': 127} =====
10 insert strategy: left_edge
11 FHInsert returned: 0
12 [RECODE] update interval: [0, 4611686018427387904)
13 [RECODE] rows to refresh with FHUpdate: 127
14 [RECODE] FHUpdate executed
15 [ENCODING UPDATED] existing rows changed: 126
16 [gap stats after step 127] min encoding gap = 36312488334073920, between row 1 and row 2
17
18 [SPLIT EXPECTED] step 128 reaches M=128; LeafNode::rebalance should be triggered in the UDF
19
20 ===== step 128: plaintext=apple, pos=0, local_table={'apple': 128} =====
21 insert strategy: left_edge
22 FHInsert returned: 0
23 [RECODE] update interval: [0, 4611686018427387904)
24 [RECODE] rows to refresh with FHUpdate: 128
25 [RECODE] FHUpdate executed
26 [ENCODING UPDATED] existing rows changed: 127
27

```

源码中 LeafNode::insert 包含如下判断：

```

1 long long LeafNode::insert(int pos, string cipher)
2 {
3     this->cipher.insert(this->cipher.begin() + pos, cipher);
4     this->encoding.insert(this->encoding.begin() + pos, -1);
5     long long cd = this->Encode(pos);
6     if (this->cipher.size() >= M)
7     {
8         this->rebalance();
9     }
10    return cd;
11 }
12

```

由于 $M=128$ ，当第 128 次插入后叶子结点中密文数量达到阈值，`LeafNode::rebalance()` 应被触发。虽然本次没有在 C++ 代码中额外加入 `cout` 打印分裂日志，但第 128 次附近达到阈值且程序继续正常插入，为分裂逻辑的执行提供了间接证据。

更明显的树结构调整证据出现在第 129 次。第 128 次以前的更新区间为：

```

1 [RECODE] update interval: [0, 4611686018427387904)
2

```

第 129 次后更新区间变为：

```

1 ===== step 129: plaintext=apple, pos=0, local_table={'apple': 129} =====
2 insert strategy: left_edge
3 FHInsert returned: 0
4 [RECODE] update interval: [0, 2305843009213693952)
5 [RECODE] rows to refresh with FHUpdate: 65
6 [RECODE] FHUpdate executed
7 [ENCODING UPDATED] existing rows changed: 64
8

```

可以看到，更新区间右端由 4611686018427387904 缩小为 2305843009213693952，受影响行数也从 128 条附近下降到 65 条。这与叶子结点分裂后的结构相符合：原来一个较大的叶子区间被拆成两个更小的区间，后续只需要对其中一段进行重新编码，而不再每次更新全部记录。

最后观察查询正确性。连续插入和多次编码更新后，最终执行 `Search('apple', 'apple')`，终端输出如下：

```

1 ===== Search result for the repeated plaintext =====
2 Search(apple, apple) returned 140 rows
3 ciphertexttext: Naoc0z2Noxb1gxXsotjGMMLsuhi/vhpqhyMTe/Z/pYYimnvbgMph7qHmilSTpeFd plaintext: apple
4 ciphertexttext: awk1SoWL/n3xb0bSNFzad3wqnPnWbT0YgwS1SsEWQceJ8fXbMBsCOHNWSUhRkw+g plaintext: apple
5 ciphertexttext: MfPPQUqWJCqQB7390x4wDkRGTAxy+VLm1Bs5ii+VzdkSEQQ9nLDZJtYz45MJxQV2 plaintext: apple
6 ...
7 ciphertexttext: 2SSv1T0etEV79d7MbD0IZDJ1joKAM5GZs6nQpazLkT2PwV9QHSdZxkQg5SzQvpqm plaintext: apple
8

```

最终返回 140 行，且每行密文解密后均为 `apple`。这说明即使中间发生了多次 `Recode` 和 `FHUpdate`，

FH-OPE 仍然保持了正确的顺序查询语义。

3.4 结论

通过本轮实验，可以得到以下结论：

- 连续插入相同明文时，客户端的 `local_table` 准确记录了相同明文出现频数。本次实验中 `apple` 最终插入 140 次。
- 随机化加密使每一次 `apple` 得到的密文不同，数据库中无法直接通过密文判断哪些记录来自同一明文。
- 采用 `left_edge` 策略后，局部编码空间被快速压缩，实验中稳定观察到了 `FHInsert returned: 0`、`Recode` 和 `FHUpdate`。
- `[ENCODING UPDATED] existing rows changed` 和多组 `old -> new` 输出说明数据库中已有记录的编码确实发生了更新。
- 第 128 次插入达到叶子结点容量阈值 `M=128`。第 129 次后更新区间明显缩小，说明编码树结构已经发生调整，符合叶子结点分裂后的表现。
- 最终查询返回 140 条 `apple` 记录，说明编码树分裂和编码更新没有破坏 FH-OPE 的范围查询正确性。

4 结论与实践心得

本次实验完成了频率隐藏 OPE 方案的复现，并在基础复现之外进一步构造了相同数值多次插入的压力测试。通过服务端 MySQL UDF 与客户端 Python 程序的配合，实验完整展示了 FH-OPE 在插入、查询、编码更新和编码树维护方面的基本运行过程。

普通 OPE 虽然能够支持密文上的顺序比较和范围查询，但相同明文通常会暴露相同或高度相关的编码模式，攻击者可能通过频率分析推断明文分布。FH-OPE 试图缓解这一问题：客户端保存频数信息，将相同明文的多次出现映射到一个位置区间中；服务端只维护密文和顺序编码，不直接获得明文频率。实验中连续插入 140 次 `apple` 时，每条记录的密文都不同，最终仍能按编码顺序查询出全部记录，这体现了频率隐藏 OPE 在功能性和隐私性之间的折中。

本次实验加深了我对可查询加密方案工程实现复杂性的理解。FH-OPE 不仅需要密码学层面的随机加密，还需要客户端状态、数据库 UDF、编码树结构和 SQL 更新逻辑协同工作。尤其是在相同值多次插入场景下，编码更新和树分裂是保证顺序查询正确性的核心机制。通过将存储过程中的逻辑展开到客户端并增加调试输出，可以更清楚地观察到算法内部状态变化，也更容易定位实验现象与源码逻辑之间的对应关系。

总体而言，本实验成功复现了 FH-OPE 方案，并通过连续插入相同明文验证了频率隐藏、编码更新、编码树分裂和范围查询正确性。实验结果表明，该方案能够在隐藏相同明文频率特征的同时保留顺序查询能力，但代价是客户端需要维护频数状态，服务端在编码空间不足时需要执行局部重编码。因此，在实际系统中使用此类方案时，需要在安全性、查询能力、更新开销和实现复杂度之间进行权衡。